STAT2/37815 | Lab # 2: Base R Fundamentals | Israel Brown Jr
Dr. McShane | | Due Sep 25th at 5pm

FOR THE ENTIRETY OF THIS ASSIGNMENT, DO NOT LOAD A PACKAGE.

.....

2.1 Vectors: `rep` function

Explore the `help` page for the `rep` function. For each of the following questions, use only one call to `rep`.

2.1.1 Use `rep` to create a vector, `vec100`, with 100 elements which alternate between 1 and 2, starting with 1.

```
vec100 = rep(c(1,2), 50)
```

2.1.2 Use `rep` to create a vector, `vec12`, which alternates between 1 and 2 100 times, starting with 1.

```
vec12 = rep(c(1,2), 100)
```

2.1.3 Use `rep` to create a vector, `vec111222`, repeats 1 100 times and then 2 100 times.

```
vec111222 = rep(c(1,2), c(100,100))
```

2.1.4 Use `rep` to create a vector, `vec112211222`, repeats 1 5 times and then 2 5 times, repeating all of this 5 times.

```
vec112211222 = rep(c(1,2), each = 5, times = 5)
```

2.2 Object Initialization

Read the help page on `vector`, `matrix`, `list`, and `data.frame`.

2.2.1 Initialize a vector of length 4 called `vec4`.

```
vec4 = vector(length = 4)
```

2.2.2 Initialize a character vector of length 8 called `char_vec8`.

```
char_vec8 = vector(mode = "character", length = 8)
```

2.2.3 Initialize an integer matrix with 4 rows and 2 columns called `int_mat42`. Each entry should be 0. Do not use `rep` or `c` or any other function to create a vector.

```
int_mat42 = matrix(data = 0L, nrow = 4, ncol=2)
```

2.2.4 Initialize an empty list of length 3 and call it `list3`.

```
list3 = vector(mode = "list", length = 3)
```

2.2.5 Using only one left-assign operator (not including `=` within a function), create a data frame called `df_1aA` with three columns called, `i`, `a`, and `A`, where each column, respectively, contains the first 26 positive integers (ascending), the lowercase letters, and the uppercase letters. Print only the first 4 rows using the `head` function.

```
df_1aA = data.frame(i = 1:26, a = letters[1:26], A = LETTERS[1:26])
head(df_1aA, n = 4)
```

```
  i a A
1 1 a A
2 2 b B
3 3 c C
4 4 d D
```

2.2.6 Create a new matrix called `mat_not_vec` which is derived from `int_mat42` – get just the first column of the matrix. Your code must be of the form `int_mat42[___]`, on a single line, and not use coercion (e.g., no usage of `as.matrix`). You will find Section 4.1.1 of HoPR helpful.

```
mat_not_vec = int_mat42[,1, drop = F]
```

2.3 Function-writing (Beginner)

2.3.1 Write a function, `alpha_beta`, which takes no arguments and returns the full lowercase alphabet in a single *string* (a character vector with a single element – has `length` one).

```
alpha_beta = function(){
  return(
    paste0(letters, collapse = "")
  )
}
```

2.3.2 Write a function, `pseudo_rand_int`, which takes two arguments: `seed`, which sets a default seed of `27815`, and `n`, which sets the upper bound for integers (1 through n) from which you `sample`. The function should return a single "random" integer and the default `n` should be 10. `pseudo_rand_int()` should return `3` and `pseudo_rand_int(seed = 37815, n = 23)` should return `13`.

```
pseudo_rand_int = function(seed = 27815, n = 10){
  set.seed(seed)
  return(sample(1:n, size = 1))
}
```

2.3.3 Write a function, `timed_rand_int`, which is identical to `psuedo_ran_int`, except the default argument for `seed` is set to `Sys.time() |> as.integer()`, which takes the current time and converts it to time in seconds. Add two calls to `timed_rand_int()` to your code chunk and see what happens when you run them simultaneously (run the code chunk a few times over a few seconds)!

```
timed_rand_int = function(seed = as.integer(Sys.time()), n = 10) {
  set.seed(seed)
  return(sample(1:n, size = 1))
}

timed_rand_int()
```

```
[1] 5
```

```
timed_rand_int()
```

```
[1] 5
```

2.3.4 Write a function, `roll20`, which takes no inputs and returns the roll of a 20-sided die *once*. Do not set a seed inside of the function.

```
roll20 = function(){
  return(sample(20, size =1))
}
```

2.3.5 Write a function, `div_by_five`, which counts how many values in a vector are divisible by 5, and takes the argument `x`, a numeric vector. Do not use a `for` or `while` loop.

```
div_by_five = function(x) {
  modulus = ((x %% 5) == 0)
  return(sum(modulus))
}
```

2.3.6 Write a function, `div_by_seven_first`, that identifies the first value of the vector that is divisible by 7, and takes the argument `x`, a numeric vector. Note: you may not use `which`. You may assume that `x` contains at least one value divisible by 7. Hint: use `min`. Do not use a `for` or `while` loop.

```
div_by_seven_first = function(x){
  modulus = ((x %% 7) == 0)
  divid = x[modulus]
  return(divid[1])
}
```

2.3.7 Write a function, `mean_my`, that calculates the arithmetic mean of a user-supplied numeric vector, `x`. Do not use the `mean` function (or any other function that calculates the mean). Do not use a `for` or `while` loop.

```
mean_my = function(x){
  return(sum(x) / length(x))
}
```

2.4 Control Flow

2.4.1 Earlier, you wrote the function `roll20`. Now, write an `if` statement which prints "that roll was even" if the output of `roll20` was even (use `paste` for printing). Before your `if` statement, set the seed to be `1003`.

```
set.seed(1003)
if ((roll20() %%2) == 0){
  paste("that roll was even")
}
```

```
[1] "that roll was even"
```

2.4.2 Write an `if` then `else` statement which prints "that roll was even" if the output of `roll20` was even and "that roll was odd" if the output of `roll20` was odd (use `paste` for printing). Before your `if`-`else` statement, set the seed to be `1001`.

```
set.seed(1001)
if ((roll20() %%2) == 0){
  paste("that roll was even")
}else {
  paste("that roll was odd")
}
```

```
[1] "that roll was odd"
```

2.4.3 Now, use the `ifelse` function to print "that roll was even" if the output of `roll20` was even and "that roll was odd" if the output of `roll20` was odd (use `paste` for printing). Before your `ifelse` function call, set the seed to be `1002`.

```
set.seed(1002)
ifelse(((roll20() %%2) == 0),
       paste("that roll was even"),
       paste("that roll was odd"))
```

```
[1] "that roll was even"
```

2.4.4 Use a `for` loop to roll the die 10 times. Copy your `ifelse` code from Problem **2.4.3** to conduct the rolls and call out the even/odd output as before. Make sure to initialize your output in a vector called `ten_rolls` and set the seed to be `1010` in the first line of code.

```
set.seed(1010)
ten_rolls = vector(mode = "character", length = 10)
for( i in seq(ten_rolls)){
  ten_rolls[i] = ifelse(
    ((roll20() %%2) == 0),
    paste("that roll was even"),
    paste("that roll was odd"))
}
```

2.4.5 Set the seed to be `1010` as you did for the previous problem. Use a call to `replicate` to roll the die 10 times and store the numeric output of the rolls in `numeric_rolls`. Copy your `ifelse` code from Problem **2.4.3** to conduct the rolls and call out the even/odd output as before, but replace `roll20()` with `numeric_rolls`. Store your even/odd callouts in a vector called `new_ten_rolls`. Check whether the output of `ten_rolls` and `new_ten_rolls` are equivalent, element-wise, and store the checks in a vector called `roll_checks`. Then, find the sum of `roll_checks` and store that in an object called `check_sum`. What is the value of `check_sum`?

```
set.seed(1010)
numeric_rolls = replicate(10, roll20())
new_ten_rolls = ifelse(
  ((numeric_rolls%%2) == 0),
  paste("that roll was even"),
  paste("that roll was odd"))

roll_checks = (ten_rolls == new_ten_rolls)

check_sum = sum(roll_checks)

paste("Value of check_sum is:", check_sum)
```

```
[1] "Value of check_sum is: 10"
```